

DevOps Phase 1 Practical Exam

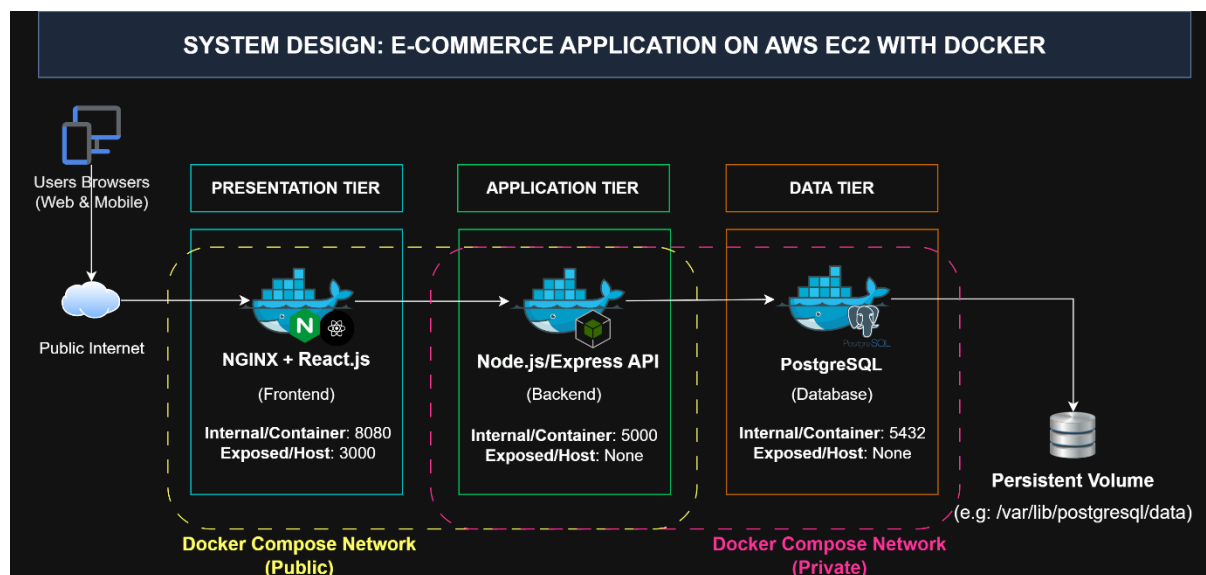
Build and deploy a 3-tier app using real DevOps practices

Project Objective:

1. This project is a fork of <https://github.com/HARSHisawesome2/fullstack-ecommerce-app>.
2. Technology Stack:
 - a. **React.js** (frontend)
 - b. **Node.js/Express.js** (backend)
 - c. **PostgreSQL** (database)

Goal: Dockerize and deploy this 3-tier e-commerce app to AWS EC2.

System Design:



1. Presentation Tier:

- a. Exposed to the users or the public internet which delivers web pages.
- b. **React.js** as a frontend module which delivers static webpages to the internet.
- c. **Nginx** used as a reverse proxy which sits between the internet and the backend server and accepts incoming web traffic then forwards it to the appropriate internal service.
- d. Ports defined:
 - i. **Host:** 3000 (Exposed to the internet to view the application)
 - ii. **Container:** 8080 (For **nginx** for **container-to-container** communication to serve web pages)

2. Application Tier:

- a. The backend application tier which delivers data through web pages.
- b. **Express API** acts as a middleware which process incoming requests and outgoing responses through the application.
- c. Ports defined:

- i. **Host:** None (Not exposed. Handled by **nginx** server)
- ii. **Container:** 5000 (For containers communication to handle data and forward it to the end user.)

3. Data Tier:

- a. Isolated database which handles user's sensitive data.
- b. **PostgreSQL** database which holds application data and **never exposed** to the internet.
- c. Ports defined:
 - i. **Host:** None (Not exposed. Isolated to the internet)
 - ii. **Container:** 5432 (For containers communication which stores user's and application data.)

How containers communicate talk to each other?

1. They communicate through docker networking. Best practice is using **User-Defined Bridge Networks**.
2. In the above diagram, there are two user-defined networks:
 - a. **Public:** It delivers application interface to the internet. The containers communicating are **React.js** and **Express.js** to orchestrate data on web pages.
 - b. **Private:** Isolated from the end-user which holds data in the database. The containers communicating are **Express.js** and **PostgreSQL** by using asynchronous **Node.js** database driver methods or ORM queries inside its route handlers to send and fetch data.

Where does data persist?

1. **Docker Volumes** is used for persisting data outside the container.
2. It stores data in a volume so that even if the container is stopped, updated or destroyed, the data is securely saved in volume.
3. In this case, the Postgres data is stored in the path, **/var/lib/postgresql/data**.

Public vs Internal (Private) on EC2

Connected directly to the internet.	Operates only within a virtual network.
Example: Web Application (React.js and Node.js)	Example: Database and backend APIs (Express API and PostgreSQL)
Port: 3000	Ports: 5000 and 5432

Deploying to AWS EC2 Instance:

Following are the steps to deploy this application on EC2 instance:

1. Login into <https://console.aws.amazon.com/> as a root user.
2. Search **EC2** from the search bar.
3. Click **Launch an instance** and fill in the details like, Amazon Machine Image (AMI), instance type, and root storage.

- Go to your created instance -> Connect to instance (Make sure your instance is ready and running) then **In SSH client** section follow the instructions.

```
ubuntu@ip-172-31-0-255: ~  
C:\Users\vishakha\OneDrive\Desktop\Phase-1-Practical-Exam>chmod 400 "fashionista-server-key.pem"  
C:\Users\vishakha\OneDrive\Desktop\Phase-1-Practical-Exam>ssh -i "fashionista-server-key.pem" ubuntu@ec2-44-243-67-131.us-west-2.compute.amazonaws.com  
The authenticity of host 'ec2-44-243-67-131.us-west-2.compute.amazonaws.com (44.243.67.131)' can't be established.  
ED25519 key fingerprint is SHA256:Eh0Azxv1CLHK6Gc65bx9RozEY257mbH2HRgbpMYGzG8.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added 'ec2-44-243-67-131.us-west-2.compute.amazonaws.com' (ED25519) to the list of known hosts.  
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)  
  
* Documentation:  https://docs.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Sat Sep  5 08:29:43 UTC 2026  
  
System load:  0.04      Processes:            111  
Usage of /:   11.0% of 18.25GB   Users logged in:     0  
Memory usage: 22%      IPv4 address for enx0: 172.31.0.255  
Swap usage:   0%  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.
```

- Update your system and install Docker and Docker Compose.

```
ubuntu@ip-172-31-0-255: ~  
ubuntu@ip-172-31-0-255:~$ sudo apt update && sudo apt install -y docker.io docker-compose-v2  
Hit:1 http://us-west-2.ec2.archive.ubuntu.com/ubuntu resolute InRelease  
Get:2 http://us-west-2.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]  
Get:3 http://us-west-2.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]  
Get:4 http://us-west-2.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 Packages [16.3 MB]  
Get:5 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]  
Get:6 http://us-west-2.ec2.archive.ubuntu.com/ubuntu resolute/universe Translation-en [6329 kB]  
Get:7 http://us-west-2.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64 Components [4556 kB]  
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64v3 Packages [472 kB]  
Get:9 http://us-west-2.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64 c-n-f Metadata [313 kB]
```

- Clone the repository for deploy and create .env file to store environment variables.

```
ubuntu@ip-172-31-0-255: ~  
ubuntu@ip-172-31-0-255:~$ git clone https://github.com/vishch24/fashionista-ecommerce-app.git  
Cloning into 'fashionista-ecommerce-app'...  
remote: Enumerating objects: 1150, done.  
remote: Counting objects: 100% (1150/1150), done.  
remote: Compressing objects: 100% (459/459), done.  
remote: Total 1150 (delta 688), reused 1114 (delta 657), pack-reused 0 (from 0)  
Receiving objects: 100% (1150/1150), 14.59 MiB | 33.87 MiB/s, done.  
Resolving deltas: 100% (688/688), done.  
ubuntu@ip-172-31-0-255:~$ cd fashionista-ecommerce-app/  
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ ls  
LICENSE.md  README.md  api  client  database-backup  docker-compose.yml  screenshots  
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ cp api/.env.example .env  
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ vim .env  
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$
```

7. Give your user to run docker commands like **docker compose**, **docker images**, etc.

```
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ whoami
ubuntu
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ sudo usermod -aG docker ubuntu
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ newgrp docker
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ docker images

INFO → U In Use

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ docker compose up -d --build

[+] Running 9/9
✔postgres Pulled 5.1s
✔b58169c5d1b0 Pull complete 0.2s
✔213ec9aee27d Pull complete 0.7s
✔48ab741cca09 Pull complete 0.7s
✔c3636f308d36 Pull complete 0.8s
✔f9b93ds592d77 Pull complete 0.8s
✔38e1167f56e0 Pull complete 0.4s
✔5da721964db9 Pull complete 0.4s
✔f4263eacea0e Pull complete 0.1s

WARN[0005] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 2.1s (4/11) docker:default
=> [node-express-api internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 1.33kB 0.0s
[=> [node-express-api internal] load metadata for docker.io/library/node:16.14.2-alpine 1.0s
=> [node-express-api internal] load .dockerignore 0.0s
=> => transferring context: 827B 0.0s
[=> [node-express-api internal] load build context 0.1s
=> => transferring context: 399.50kB 0.0s
[=> [node-express-api builder 1/5] FROM docker.io/library/node:16.14.2-alpine@sha256:28bed508446db2ee028d08e76fb47b935defa26a84986ca050d2596ae67fd506 0.9s
=> => resolve docker.io/library/node:16.14.2-alpine@sha256:28bed508446db2ee028d08e76fb47b935defa26a84986ca050d2596ae67fd506 0.0s
=> => sha256:7da04ed7d1ef87b97971adcd1774727fbd83f778a1b8d9b989869d4d4d6f999f4 449B / 449B 0.0s
=> => sha256:f7c8a32a53f2d6f52529b16b4f815394ead5dbebe109799da58d6cfc407189245 0B / 2.34MB 0.8s
=> => sha256:62212590d8a369b76f1db3a6fd5b8d5796b08cfcd9a9f765c39d4b180e5854 32.51MB / 35.40MB 0.7s
=> => sha256:d43b938940a46279a7418b85cedfdbc2288c0a083da7ab5613af71079148139 2.81MB / 2.81MB 0.3s
=> => extracting sha256:f9b9388f04ad6279a7418b85cedfdbc2288c0a083da7ab5613af71079148139 0.3s
```

8. Check your containers are **running** and **healthy**.

```
[*] Running 8/8
✔node-express-api Built 0.0s
✔react-client Built 0.0s
✔Network fashionista-app_fashionista-public Created 0.1s
✔Network fashionista-app_fashionista-private Created 0.1s
✔Volume fashionista-app_pgdata Created 0.0s
✔Container fashionista-app-postgres-1 Healthy 6.4s
✔Container fashionista-app-node-express-api-1 Healthy 37.8s
✔Container fashionista-app-react-client-1 Started 37.2s

ubuntu@ip-172-31-0-255:~/fashionista-e-commerce-app$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
869e1090219e   fashionista-app-react-client        "nginx -c /etc/nginx. ..." About a minute ago Up 26 seconds (health: starting) 0.0.0.0:3000->8080/tcp, [
::]:3000->8080/tcp
8169821f59cc   fashionista-app-node-express-api    "docker-entrypoint.s. ..." About a minute ago Up 57 seconds (healthy) 5080/tcp
be25b8f224dc   postgres:14.5-alpine                "docker-entrypoint.s. ..." About a minute ago Up About a minute (healthy) 5432/tcp
fashionista-app-postgres-1

ubuntu@ip-172-31-0-255:~/fashionista-e-commerce-app$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
869e1090219e   fashionista-app-react-client        "nginx -c /etc/nginx. ..." About a minute ago Up 44 seconds (healthy) 0.0.0.0:3000->8080/tcp, [::]:3
8080->8080/tcp
8169821f59cc   fashionista-app-node-express-api    "docker-entrypoint.s. ..." About a minute ago Up About a minute (healthy) 5080/tcp
be25b8f224dc   postgres:14.5-alpine                "docker-entrypoint.s. ..." About a minute ago Up About a minute (healthy) 5432/tcp
fashionista-app-postgres-1

ubuntu@ip-172-31-0-255:~/fashionista-e-commerce-app$ |
```

9. Check image sizes and disk usage.

```
ubuntu@ip-172-31-0-255:~/fashionista-e-commerce-app$ docker compose images
```

CONTAINER	REPOSITORY	TAG	PLATFORM	IMAGE ID	SIZE	CREA
ted						
fashionista-app-node-express-api-1	fashionista-app-node-express-api	latest	linux/amd64	946fdceb4aa2	46.7MB	5 mi
nutes ago						
fashionista-app-postgres-1	postgres	14.5-alpine	linux/amd64	ac09c433f64f	84.7MB	5 mi
nutes ago						
fashionista-app-react-client-1	fashionista-app-react-client	latest	linux/amd64	f97757f2d9e9	23.7MB	2 mi
nutes ago						

```
ubuntu@ip-172-31-0-255:~/fashionista-e-commerce-app$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
fashionista-app-node-express-api:latest	946fdceb4aa2	211MB	46.7MB	U
fashionista-app-react-client:latest	f97757f2d9e9	84.8MB	23.7MB	U
postgres:14.5-alpine	ac09c433f64f	311MB	84.7MB	U

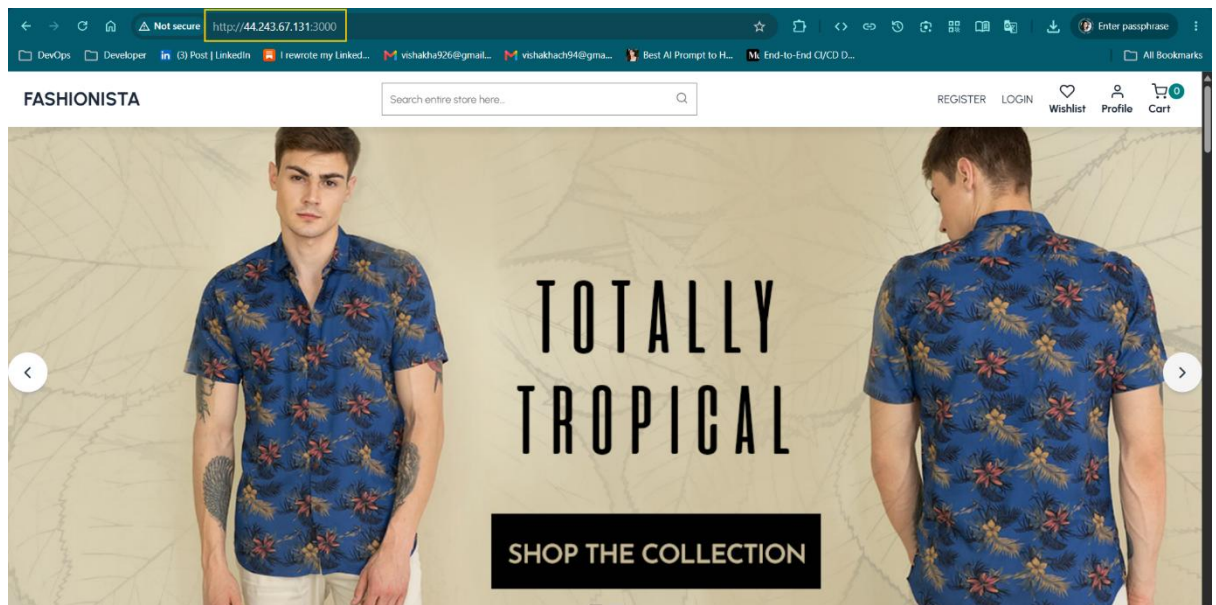
```
ubuntu@ip-172-31-0-255:~/fashionista-e-commerce-app$
```

10. Go to your instance and set **inbound rules (incoming traffic)** in security group.

The screenshot shows the AWS Management Console 'Instances' page. The instance 'fashionista-ec2' (ID: i-0ea147e7fdc12b70d) is highlighted. Below the instance list, the 'Security group rule ID' section is expanded, showing four inbound rules. The 'Port range' column is highlighted with a yellow box.

Name	Security group rule ID	Port range	Protocol	Source	Security groups
-	sgr-03acde7e442c62379	80	TCP	0.0.0.0/0	launch-wizard-16
-	sgr-06c4428685999351b	3000	TCP	0.0.0.0/0	launch-wizard-16
-	sgr-02d73dbae21a10287	22	TCP	0.0.0.0/0	launch-wizard-16
-	sgr-074cae2e793c8e65	443	TCP	0.0.0.0/0	launch-wizard-16

11. Access your application by <http://<public-ec2-ip-address>:<exposed-port>>.



12. Check tables and schema of the **PostgreSQL** database using **docker compose exec** command.

```
ubuntu@ip-172-31-0-255: ~/f. X + v
ubuntu@ip-172-31-0-255:~/fashionista-ecommerce-app$ docker compose exec postgres psql -U ecommerce -d ecommerce
psql (14.5)
Type "help" for help.

ecommerce=# \dt
          List of relations
Schema | Name          | Type  | Owner
-----+-----+-----+-----
public | addresses     | table | ecommerce
public | cart_items    | table | ecommerce
public | carts         | table | ecommerce
public | categories    | table | ecommerce
public | order_items   | table | ecommerce
public | orders        | table | ecommerce
public | product_carts | table | ecommerce
public | product_categories | table | ecommerce
public | product_sizes | table | ecommerce
public | product_variants | table | ecommerce
public | products      | table | ecommerce
public | productsizes  | table | ecommerce
public | shippingaddresses | table | ecommerce
public | sizes         | table | ecommerce
public | tokens        | table | ecommerce
public | users         | table | ecommerce
(16 rows)

ecommerce=# \d users
          Table "public.users"
  Column          | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
id                | integer       |           | not null | nextval('users_id_seq'::regclass)
firstName         | character varying(20) |           | not null |
lastName          | character varying(20) |           | not null |
username          | character varying(20) |           | not null |
email             | character varying(40) |           | not null |
hashedPassword    | character varying(64) |           | not null |
isAdmin           | boolean       |           | not null |
createdAt         | timestamp with time zone |           | not null |
updatedAt         | timestamp with time zone |           | not null |
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
```

13. Check seeded data.

```
ecommerce=# SELECT count(*) FROM products;
 count
-----
    68
(1 row)

ecommerce=# SELECT username, email FROM users;
 username | email
-----+-----
testaccount | test@test.com
(1 row)

ecommerce=# SELECT id, category FROM categories;
 id | category
-----+-----
  7 | Men's Shirt
  8 | Men's T-Shirt
 12 | Hoodies
  9 | Saree
 10 | Lehenga
 11 | Jackets & Cardigans
 13 | Women's Night Suit
(7 rows)

ecommerce=# |
```


14. Register through the application to test backend.

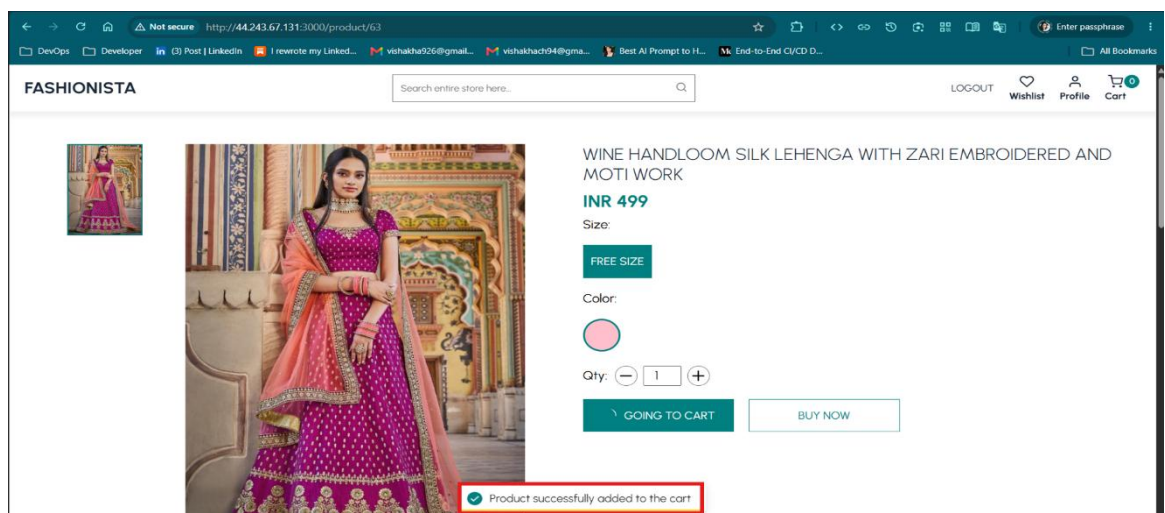
The screenshot shows the 'FASHIONISTA' website's registration page. The browser address bar shows 'http://44.243.67.131:3000/register'. The page has a search bar and navigation links for 'REGISTER', 'LOGIN', 'Wishlist', 'Profile', and 'Cart'. The main content area is titled 'Create New Account' and contains a form with the following fields: 'Vishakha' (first name), 'Chavan' (last name), 'vishakhach' (username), 'vishakhach@example.com' (email), and two password fields. A 'SIGN UP' button is at the bottom of the form, with a link 'Already Have an Account? Login Here' below it. To the right of the form is an illustration of a person in a hijab carrying a box, with a display of clothing items (pants, dress, shirt, boots, t-shirt) above them.

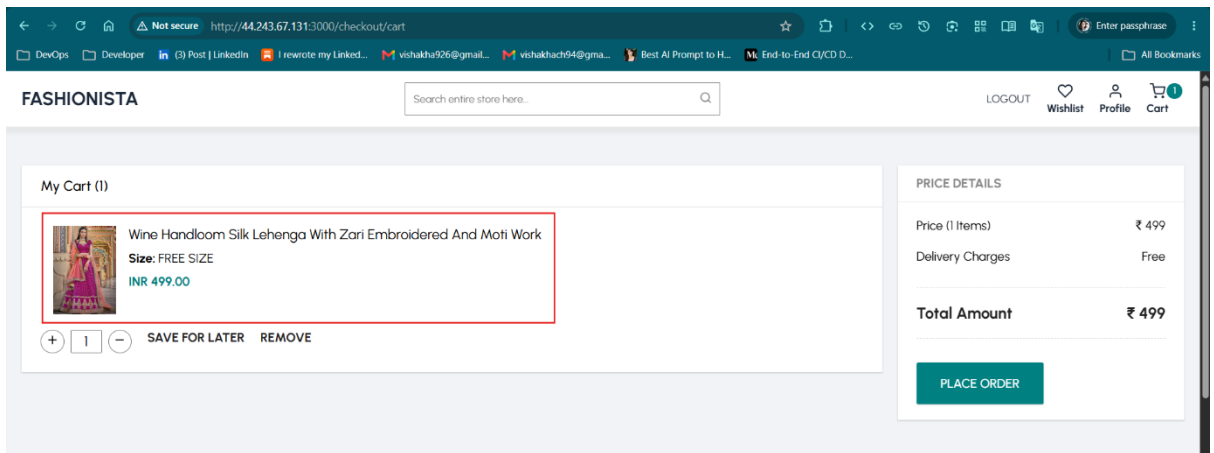
15. Data saved in the database.

```
ecommerce=# SELECT username, email FROM users;
username | email
-----+-----
testaccount | test@test.com
vishakhach | vishakhach@example.com
(2 rows)

ecommerce=# |
```

16. Check **Add to cart** functionality.





17. Check data persistence after the containers are stopped and started again.

```

ubuntu@ip-172-31-0-255: ~/fashionista-ecommerce-app$ docker compose down
[+] Running 5/5
  ✓ Container fashionista-app-react-client-1 Removed 0.3s
  ✓ Container fashionista-app-node-express-api-1 Removed 10.2s
  ✓ Container fashionista-app-postgres-1 Removed 0.2s
  ✓ Network fashionista-app_fashionista-private Removed 0.1s
  ✓ Network fashionista-app_fashionista-public Removed 0.1s
ubuntu@ip-172-31-0-255: ~/fashionista-ecommerce-app$ docker compose up -d
[+] Running 5/5
  ✓ Network fashionista-app_fashionista-private Created 0.1s
  ✓ Network fashionista-app_fashionista-public Created 0.1s
  ✓ Container fashionista-app-postgres-1 Healthy 5.9s
  ✓ Container fashionista-app-node-express-api-1 Healthy 36.6s
  ✓ Container fashionista-app-react-client-1 Started 36.8s
ubuntu@ip-172-31-0-255: ~/fashionista-ecommerce-app$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS
11f4c71ac5c3   fashionista-app-react-client-1      "nginx -c /etc/nginx.s..." About a minute ago Up About a minute (healthy) 0.0.0.0:3000->8080/tcp, [::]:3
90fb8f05ea67   fashionista-app-node-express-api-1  "docker-entrypoint.s..." About a minute ago Up About a minute (healthy) 5000/tcp
60d2b2a1f395   postgres:14.5-alpine               "docker-entrypoint.s..." About a minute ago Up About a minute (healthy) 5432/tcp
ubuntu@ip-172-31-0-255: ~/fashionista-ecommerce-app$ docker compose exec postgres psql -U ecommerce -d ecommerce
psql (14.5)
Type "help" for help.

ecommerce=# SELECT username, email FROM users;
 username | email
-----
testaccount | test@test.com
vishakhach | vishakhach@example.com
(2 rows)

ecommerce=#

```

Problems I faced

1. Native module compilation failure on Alpine Linux:

bcrypt compiles a C++ binding via **node-gyp** at install time. Alpine uses **musl** libc instead of **glibc**, and prebuilt binaries for **bcrypt** on **musl** + Node 16 are not reliably available. Running **npm ci** on a plain **node:16.14.2-alpine** image failed with a compiler-not-found error.

2. Database seeding given as a binary dump:

The dump was in PostgreSQL binary format. It couldn't simply be dropped into **docker-entrypoint-initdb.d** as a **.sql** file.

How I solved them?

1. Added build toolchain installation before **npm ci** in the API Dockerfile builder stage:
RUN apk add --no-cache build-base g++ make python3.
2. Used multi-stage build so the compiler tools (**python3, make, g++**) never travel into the production image. Only the compiled **node_modules** are copied to the runtime stage.
3. **Result:** smaller, more secure production image with no build tools in the attack surface.
4. Created **database-backup/restore.sh** using **pg_restore** with **--no-owner** and **--no-privileges** flags to handle the original developer's postgres username not existing in the container.
5. Mounted both the script and the binary dump directly into **docker-entrypoint-initdb.d** via compose volume mounts.
6. PostgreSQL automatically executes **.sh** files in that directory on first boot with a fresh volume.
7. Verified with full reproducibility test: **docker compose down -v** followed by **docker compose up -d** produces 68 products with zero manual intervention.

What I learned?

1. Docker Fundamentals

- a. **ARG** declared before **FROM** only scopes to that **FROM** line. Re-declare it inside the stage to use it in **RUN** or **ENV** instructions.
- b. Multi-stage builds aren't just for size reduction. They isolate build tools (**compilers, python3, make**) from the production runtime image, shrinking the attack surface.

2. Environment Variables Importance

- a. **env_file:** in compose passes all variables from **.env** to the container. Without it, only explicitly listed **environment:** keys exist inside the container, everything else is **undefined**.
- b. Build-time vars (**REACT_APP_***) must be declared as **ARG/ENV** before **RUN npm run build** in the Dockerfile. **Create React App (CRA)** bakes them into the static JS bundle at build time, not runtime.

3. Nginx Role as a Reverse Proxy

- a. **proxy_pass http://container-name:port** uses Docker's internal DNS. The browser never touches the backend port directly. All API traffic flows through nginx on the single public port.
- b. Trailing slash on **proxy_pass** changes path forwarding behavior. Without it nginx forwards the full original URI. With it, nginx strips the matched location prefix before forwarding.

4. Data Persistence Using Docker Volume

- a. **docker compose down** preserves named volumes. **docker compose down -v** destroys them. Testing both scenarios separately is the only way to prove true reproducibility.

Conclusion

This project is complete after taking a raw developer codebase with zero containerization and transforming it into a production-ready, reproducible, and secure 3-tier Dockerized application which was deployed on AWS EC2.

The journey covered far more than just writing Dockerfiles. It required reading and understanding someone else's code, identifying bugs that only surface in containerized environments, making deliberate architectural decisions, and debugging real failures across networking, security, environment configuration, and database connectivity. The exact class of problems a DevOps engineer faces daily.

All source code, Dockerfiles, compose configuration, nginx config, database restore script, and documentation are available at:

<https://github.com/vishch24/fashionista-ecommerce-app>

A heartfelt thank you to **Shubham Londhe** and the entire **TrainWithShubham** community for designing a practical exam that actually teaches rather than just tests. The structured roadmap across Linux, Git, Networking, System Design, and Docker gave every concept a real context to land in, not theory in isolation, but skills applied to a problem that fights back.